
Security Review Report

NM-0988 - Vistapex



NETHERMIND
SECURITY

(August 21, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	4
4.1	DepositPool	4
4.1.1	Deposits	4
4.1.2	Withdrawals	4
4.2	PerpsStateVerifier	4
4.3	Administrative Controls and Trust Assumptions	4
5	Risk Rating Methodology	5
6	Issues	6
6.1	[Info] Calling pause(...) and unpause(...) without checking current state leads to redundant event emissions	6
6.2	[Info] Defensive balance-check pattern in deposit() is unnecessary for standard USDC	6
6.3	[Info] Ownership transfer does not revoke pending operational-role nominations across both contracts	6
6.4	[Info] Role rotation functions do not check that the new address differs from the current one	6
6.5	[Best Practices] Missing enforcement of distinct addresses between owner and operator/authorizer roles	7
7	Documentation Evaluation	8
8	Test Suite Evaluation	9
8.1	Tests Output	9
9	About Nethermind	12

1 Executive Summary

This document presents the results of the security review conducted by [Nethermind Security](#) for [Vistapex](#) smart contract.

The audit comprises 484 lines of Solidity code. **The audit was performed using** (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report five points of attention, where four are classified as Informational, and one is classified as Best Practices.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 4 presents the System Overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

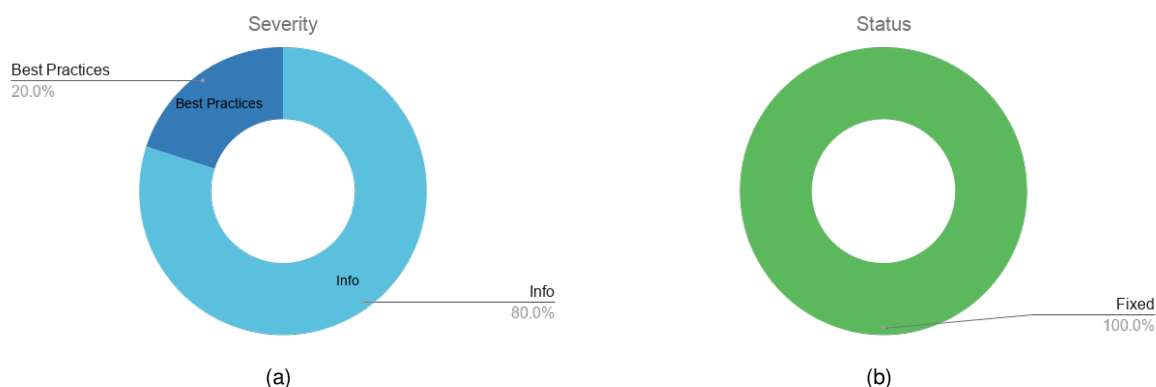


Fig. 1: Distribution of issues: Critical (0), High (0), Medium (0), Low (0), Undetermined (0), Informational (4), Best Practices (1).
Distribution of status: Fixed (5), Acknowledged (0), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	August 21, 2026
Final Report	August 21, 2026
Initial Commit	c22dd9cbd1c
Final Commit	9da7c5ffca
Documentation Assessment	Medium
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	PerpsStateVerifier.sol	232	273	117.7%	54	559
2	DepositPool.sol	252	213	84.5%	67	532
	Total	484	486	100.4%	121	1091

3 Summary of Issues

	Finding	Severity	Update
1	Calling pause(...) and unpause(...) without checking current state leads to redundant event emissions	Info	Fixed
2	Defensive balance-check pattern in deposit() is unnecessary for standard USDC	Info	Fixed
3	Ownership transfer does not revoke pending operational-role nominations across both contracts	Info	Fixed
4	Role rotation functions do not check that the new address differs from the current one	Info	Fixed
5	Missing enforcement of distinct addresses between owner and operator/authorizer roles	Best Practices	Fixed

4 System Overview

VistaPex is a perpetual futures system deployed on Avalanche. The in-scope onchain layer consists of two independent contracts. **DepositPool** provides pooled USDC custody and settlement, while **PerpsStateVerifier** records SP1-proven state commitments for transparency. The contracts do not call each other or share state. Offchain services are assumed to maintain trading accounts, authorize settlements, produce proofs, and consume contract events.

4.1 DepositPool

The **DepositPool** contract holds the protocol's USDC and exposes deposit and withdrawal settlement functions. The USDC address is immutable. The contract does not maintain user balances or a per-account ledger, so trading balances and account ownership are assumed to be maintained by the offchain system. Its onchain records consist primarily of custody balances, replay-protection mappings, configuration, and events.

4.1.1 Deposits

A user first approves the pool to spend USDC and then calls `deposit` with a non-zero trading account ID and amount. The contract measures its token balance before and after `transferFrom` and emits a `Deposit` event for the amount actually received. This prevents the offchain system from crediting more than reached the pool if the configured token has non-standard transfer behavior. An assumed chain watcher consumes the event after confirmation and credits the corresponding offchain account. As `deposit` is a public function, anyone can call this function directly on the smart contract. Any funds deposited with an invalid trading account ID will remain permanently locked in the contract.

4.1.2 Withdrawals

Withdrawals use separate authorizer and operator roles. The authorizer signs the settlement parameters offchain, while the operator is the only address allowed to submit `withdraw`. The signed message commits to the pool address, withdrawal ID, recipient, amount, nonce, deadline, and chain ID, which binds the authorization to one deployment and network.

The contract requires a non-zero amount and recipient, an unexpired deadline, an unused nonce, and an unsettled withdrawal ID. It also enforces the owner-configurable per-transaction withdrawal cap when the cap is non-zero. After recovering the configured authorizer from the signature, it consumes both replay identifiers before transferring USDC and emitting `Withdrawal`. The withdrawal ID prevents the same offchain request from being settled again with a new nonce, while the nonce prevents reuse of an authorization value.

4.2 PerpsStateVerifier

The **PerpsStateVerifier** contract is a transparency component and has no authority over funds held by **DepositPool**. Its proof operator submits SP1 proof bytes together with nine ABI-encoded public values: the pre-state root, post-state root, batch ID, event count, oracle-prices hash, events hash, market-configurations hash, chain ID, and the **PerpsStateVerifier** deployment address.

The contract asks the configured SP1 verifier to validate the proof against the current program verification key. It then checks that the committed chain ID and verifier address match the current deployment, that the pre-state root equals the stored `balanceStateRoot`, and that the batch ID immediately follows `lastProvenBatchId`. A successful submission stores the post-state root and batch ID and emits `BatchProven` with the batch metadata commitments. External observers can read the latest proven state directly from the contract.

4.3 Administrative Controls and Trust Assumptions

Both contracts use two-step ownership transfers. In **DepositPool**, operator and authorizer rotations require a proposal, a 48-hour delay, and acceptance by the nominated address. The owner can also configure the withdrawal cap. The owner, operator, or authorizer may pause deposits and withdrawals, but only the owner may unpause them. In **PerpsStateVerifier**, operator, program verification key, and SP1 verifier rotations require a proposal and a 48-hour delay. The nominated operator accepts its own rotation, while the owner accepts program-key and SP1-verifier rotations. The owner or proof operator may pause proof submission, while only the owner may unpause it.

The verifier's initial state root is supplied to the constructor and is not derived or authenticated onchain. The `BatchProven` event commits to hashes of prices, events, and market configurations. The contract also provides no escape hatch that lets users exit from the proven state without cooperation from the custody and offchain settlement system.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Info] Calling pause(...) and unpause(...) without checking current state leads to redundant event emissions

File(s): [src/DepositPool.sol](#), [src/PerpsStateVerifier.sol](#)

Description: The `pause(...)` and `unpause(...)` functions in the `DepositPool` and `PerpsStateVerifier` contracts do not check whether the contract is already in the target state before updating storage and emitting state transition events.

As a result, authorized users can invoke `pause(...)` or `unpause(...)` repeatedly when the contract is already in that state, leading to duplicate `Paused` or `Unpaused` events still being emitted.

Recommendation(s): Consider adding state validation guards to both `pause(...)` and `unpause(...)` functions to verify that the contract is not already in the target state prior to performing storage modifications and emitting events.

Status: Fixed

Update from the client: [Commit #9da7c5ff](#)

6.2 [Info] Defensive balance-check pattern in `deposit()` is unnecessary for standard USDC

File(s): [src/DepositPool.sol](#)

Description: The `deposit()` function measures the actual USDC received via a `balanceOf` before/after pattern rather than trusting the caller-declared amount. However, USDC on Avalanche is a standard, non-rebasing, non-fee-on-transfer token, meaning the amount received will always equal the amount under current conditions.

USDC has an upgradeable proxy architecture, and hence Circle could theoretically introduce a transfer fee via a contract upgrade at any time in the future.

Recommendation(s): Consider introducing an owner-configurable maximum acceptable fee threshold in basis points (e.g., `maxDepositFeeBps`). On each deposit, compute the fee as the amount received and revert if it exceeds the configured threshold. This gives the team explicit control over how much transfer-fee slippage is acceptable.

Status: Fixed

Update from the client: [Commit #9da7c5ff](#)

6.3 [Info] Ownership transfer does not revoke pending operational-role nominations across both contracts

File(s): [src/DepositPool.sol](#), [src/PerpsStateVerifier.sol](#)

Description: In both `DepositPool` and `PerpsStateVerifier`, `acceptOwnership(...)` updates only owner and clears `pendingOwner`, but does not invalidate any pending operational-role rotations that the previous owner may have proposed. These stale nominations remain executable by their nominees after the handover of ownership, without any authorization from the new owner.

When ownership changes, the contract must automatically clear all pending role nominations and reset their ETAs value to zero.

Recommendation(s): Clear all pending operational-role slots and their associated ETAs during ownership acceptance.

Status: Fixed

Update from the client: [Commit #9da7c5ff](#)

6.4 [Info] Role rotation functions do not check that the new address differs from the current one

File(s): [src/DepositPool.sol](#)

Description: The `DepositPool` contract manages rotations of administrative roles for the owner, operator, and authorizer. Role changes follow a two-step pattern: the owner proposes a new address, and a timelock must elapse before the rotation can be finalized.

However, the `proposeOperator(...)`, `proposeAuthorizer(...)`, and `transferOwnership(...)` functions only validate that the proposed address is not `address(0)`. They fail to verify whether the proposed address matches the current holder of the role.

Recommendation(s): Consider adding validation in `proposeOperator(...)`, `proposeAuthorizer(...)`, and `transferOwnership(...)` to ensure that the proposed address does not match the current holder.

Status: Fixed

Update from the client: [Commit #9da7c5ff](#)

6.5 [Best Practices] Missing enforcement of distinct addresses between owner and operator/authorizer roles

File(s): [src/DepositPool.sol](#), [src/PrepsStateVerifier.sol](#)

Description: The DepositPool contract relies on a two-key custody separation model where the operator submits withdrawal transactions and the authorizer signs authorization payloads. Compromising either key alone should not be sufficient to drain funds.

The contract enforces that operator != authorizer cannot be the same address, but does not enforce the owner address to be distinct from operator or authorizer address.

As owner, operator, and authorizer are three distinct roles, it is recommended to keep them separate and not allow the roles to collapse into a single address.

A similar pattern exists in the PrepStateVerifier contract.

Recommendation(s): Consider adding explicit validation checks in the constructor(...) and all role assignment and acceptance functions, such as acceptOperator(...), acceptAuthorizer(...), and acceptOwnership(...), to ensure that the owner, operator, and authorizer variables strictly store unique addresses.

Status: Fixed

Update from the client: [Commit #9da7c5ff](#)

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Vistapex' documentation

The Vistapex team provided an overview of their Perpetuals system comprising of off-chain and on-chain components, with main focus on the on-chain smart contracts and their functionality. Additionally, the team addressed all questions and concerns raised by the Nethermind Security team, providing valuable insights and an in-depth understanding of the project's technical aspects.

8 Test Suite Evaluation

8.1 Tests Output

```

Ran 2 tests for test/MintableUSDC.t.sol:MintableUSDCDepositPoolIntegrationTest
[PASS] test_deposit_against_mintable_usdc_matches_vanilla_erc20_behaviour() (gas: 138521)
[PASS] test_withdraw_against_mintable_usdc_settles_to_recipient() (gas: 182360)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 1.45ms (685.42µs CPU time)

Ran 66 tests for test/DepositPool.t.sol:DepositPoolTest
[PASS] test_acceptAuthorizer_revertsIfNotPendingAuthorizer() (gas: 68473)
[PASS] test_acceptOperator_revertsIfNotPendingOperator() (gas: 66062)
[PASS] test_acceptOwnership_revertsIfNotPendingOwner() (gas: 39660)
[PASS] test_acceptOwnership_revokesStaleRoleNominations() (gas: 124120)
[PASS] test_authorizerRotation_fullFlow() (gas: 183422)
[PASS] test_cancelPendingAuthorizer_clearsSlot() (gas: 55811)
[PASS] test_cancelPendingAuthorizer_revertsIfNoneOrNotOwner() (gas: 75092)
[PASS] test_cancelPendingOperator_clearsSlot() (gas: 53342)
[PASS] test_cancelPendingOperator_revertsIfNoneOrNotOwner() (gas: 71593)
[PASS] test_constructor_revertsIfOperatorEqualsAuthorizer() (gas: 43983)
[PASS] test_constructor_revertsOnContractAuthorizer() (gas: 45830)
[PASS] test_constructor_revertsOnZeroAuthorizer() (gas: 42570)
[PASS] test_constructor_revertsOnZeroOperator() (gas: 42821)
[PASS] test_constructor_revertsOnZeroUsdc() (gas: 42613)
[PASS] test_constructor_setsOperatorAndAuthorizer() (gas: 16703)
[PASS] test_constructor_setsOwnerToDeployer() (gas: 8824)
[PASS] test_deposit_creditsMeasuredReceivedWhenFeeWithinTolerance() (gas: 1912528)
[PASS] test_deposit_creditsPoolAndEmitsEvent() (gas: 64796)
[PASS] test_deposit_revertsOnFeeTokenAtDefaultTolerance() (gas: 1885442)
[PASS] test_deposit_revertsOnZeroAmount() (gas: 14580)
[PASS] test_deposit_revertsOnZeroTradingAccountId() (gas: 15055)
[PASS] test_deposit_revertsWhenFeeExceedsConfiguredTolerance() (gas: 1906909)
[PASS] test_deposit_revertsWhenTransferFromFails() (gas: 1723117)
[PASS] test_operatorRotation_fullFlow() (gas: 169413)
[PASS] test_pause_blocksDeposit() (gas: 40827)
[PASS] test_pause_blocksWithdraw() (gas: 86528)
[PASS] test_pause_byAuthorizer() (gas: 42440)
[PASS] test_pause_byOperator() (gas: 43975)
[PASS] test_pause_byOwner() (gas: 34062)
[PASS] test_pause_emitsPausedAndUnpausedEvents() (gas: 27652)
[PASS] test_pause_refusesRedundantCalls() (gas: 28240)
[PASS] test_pause_revertsForStranger() (gas: 17307)
[PASS] test_poolBalance_reflectsDepositsAndWithdrawals() (gas: 151892)
[PASS] test_proposeAuthorizer_emitsSupersededOnOverwrite() (gas: 77934)
[PASS] test_proposeAuthorizer_revertsIfEqualsOperator() (gas: 16211)
[PASS] test_proposeAuthorizer_revertsIfNotOwner() (gas: 15260)
[PASS] test_proposeAuthorizer_revertsOnContractAddress() (gas: 16455)
[PASS] test_proposeAuthorizer_revertsOnZeroAddress() (gas: 11663)
[PASS] test_proposeOperator_revertsIfEqualsAuthorizer() (gas: 15705)
[PASS] test_proposeOperator_revertsIfNotOwner() (gas: 15230)
[PASS] test_proposeOperator_revertsOnZeroAddress() (gas: 11485)
[PASS] test_propose_refusesUnchangedAddresses() (gas: 32028)
[PASS] test_roles_mustStayDistinctFromOwner() (gas: 23549)
[PASS] test_setMaxWithdrawalPerTx_revertsIfNotOwner() (gas: 14243)
[PASS] test_transferOwnership_fullFlow() (gas: 91134)
[PASS] test_transferOwnership_revertsIfNotOwner() (gas: 16721)
[PASS] test_transferOwnership_revertsOnZeroAddress() (gas: 12667)
[PASS] test_twoKeyCollapse_blockedAtAccept() (gas: 93587)
[PASS] test_unpause_operatorCannot() (gas: 41712)
[PASS] test_unpause_restoresDeposit() (gas: 66476)
[PASS] test_withdrawHasDeadline_markerIsTrue() (gas: 6745)
[PASS] test_withdraw_respectsMaxWithdrawalCap() (gas: 180932)
[PASS] test_withdraw_revertsOnCrossChainReplay() (gas: 78785)
[PASS] test_withdraw_revertsOnExpiredSignature() (gas: 68777)
[PASS] test_withdraw_revertsOnHighSSignature() (gas: 74333)
[PASS] test_withdraw_revertsOnInvalidSignature() (gas: 80128)
[PASS] test_withdraw_revertsOnNonceReuse() (gas: 149466)
[PASS] test_withdraw_revertsOnSettledWithdrawalIdWithFreshNonce() (gas: 153199)
[PASS] test_withdraw_revertsOnWrongSignatureLength() (gas: 27214)
[PASS] test_withdraw_revertsOnZeroAmount() (gas: 20288)
[PASS] test_withdraw_revertsOnZeroRecipient() (gas: 21294)

```

```
[PASS] test_withdraw_revertsWhenNotOperator() (gas: 68777)
[PASS] test_withdraw_revertsWhenSignedByOperatorNotAuthorizer() (gas: 79428)
[PASS] test_withdraw_revertsWhenTransferFails() (gas: 1776497)
[PASS] test_withdraw_sendsUsdcAndEmitsEvent() (gas: 133287)
[PASS] test_withdraw_succeedsAtDeadlineBoundary() (gas: 128823)
Suite result: ok. 66 passed; 0 failed; 0 skipped; finished in 1.80ms (6.04ms CPU time)
```

```
Ran 47 tests for test/PerpsStateVerifier.t.sol:PerpsStateVerifierTest
[PASS] test_abiDecodeOrder_matchesGuestSolStructLayout() (gas: 1025723)
[PASS] test_acceptOperator_revertsOnUnauthorized() (gas: 68905)
[PASS] test_acceptOwnership_revokesAllPendingRotations() (gas: 220575)
[PASS] test_cancelPendingOperator_clearsSlot() (gas: 60760)
[PASS] test_cancelPendingProgramVKey_clearsSlot() (gas: 54866)
[PASS] test_cancelPendingSP1Verifier_clearsSlot() (gas: 120719)
[PASS] test_constructor_emitsVKeyUpdatedVersion1() (gas: 1012571)
[PASS] test_constructor_genesisInitialBatchIdZero() (gas: 31739)
[PASS] test_constructor_migrationInitialBatchIdNonZero() (gas: 1023708)
[PASS] test_constructor_revertsOnEoaVerifierAddress() (gas: 44824)
[PASS] test_constructor_revertsOnMaxUint64InitialBatchId() (gas: 47147)
[PASS] test_constructor_revertsOnZeroOperatorAddress() (gas: 42579)
[PASS] test_constructor_revertsOnZeroProgramVKey() (gas: 45060)
[PASS] test_constructor_revertsOnZeroVerifierAddress() (gas: 41016)
[PASS] test_operatorRotation_twoStepTimelockedFlow() (gas: 63192)
[PASS] test_pauseUnpauseFlow() (gas: 32891)
[PASS] test_pause_callableByOperator() (gas: 42838)
[PASS] test_pause_refusesRedundantCalls() (gas: 31566)
[PASS] test_pause_revertsOnUnauthorized() (gas: 16210)
[PASS] test_programVKeyRotation_timelockGate() (gas: 61312)
[PASS] test_proposeOperator_revertsOnUnauthorized() (gas: 13595)
[PASS] test_proposeOperator_revertsOnZeroAddress() (gas: 13560)
[PASS] test_propose_refusesUnchangedAndCollidingAddresses() (gas: 30537)
[PASS] test_setProgramVKey_preservesContinuityAcrossRotation() (gas: 87731)
[PASS] test_setProgramVKey_revertsOnUnauthorized() (gas: 20560)
[PASS] test_setProgramVKey_revertsOnUnchangedVKey() (gas: 16736)
[PASS] test_setProgramVKey_revertsOnZeroVKey() (gas: 13671)
[PASS] test_setProgramVKey_rotatesBumpsVersionAndEmits() (gas: 110157)
[PASS] test_setProgramVKey_submitVerifiesAgainstRotatedVKey() (gas: 1145072)
[PASS] test_setSP1Verifier_revertsOnUnauthorized() (gas: 99649)
[PASS] test_setSP1Verifier_revertsOnZeroOrEoaAddress() (gas: 25332)
[PASS] test_setSP1Verifier_rotatesAndRoutesCalls() (gas: 189931)
[PASS] test_submitBatchProof_chainsConsecutiveBatches() (gas: 48774)
[PASS] test_submitBatchProof_rejectsOnVerifierRevert() (gas: 1092803)
[PASS] test_submitBatchProof_revertsOnBatchIdReplay() (gas: 43591)
[PASS] test_submitBatchProof_revertsOnBatchIdSkip() (gas: 28849)
[PASS] test_submitBatchProof_revertsOnPreRootMismatch() (gas: 28902)
[PASS] test_submitBatchProof_revertsOnUnauthorized() (gas: 16137)
[PASS] test_submitBatchProof_revertsOnWrongChainId() (gas: 27816)
[PASS] test_submitBatchProof_revertsOnWrongVerifierAddress() (gas: 27306)
[PASS] test_submitBatchProof_revertsWhenPaused() (gas: 45515)
[PASS] test_submitBatchProof_updatesStateAndEmitsEvent() (gas: 44542)
[PASS] test_transferOwnership_isTwoStepAndRecoverable() (gas: 59668)
[PASS] test_transferOwnership_revertsOnUnauthorized() (gas: 15131)
[PASS] test_transferOwnership_revertsOnZeroAddress() (gas: 14681)
[PASS] test_unpause_revertsForOperator() (gas: 40518)
[PASS] test_unpause_revertsOnUnauthorized() (gas: 41079)
Suite result: ok. 47 passed; 0 failed; 0 skipped; finished in 1.76ms (2.79ms CPU time)
```

```
Ran 38 tests for test/MintableUSDC.t.sol:MintableUSDCTest
[PASS] test_acceptOwnership_revertsIfNotPending() (gas: 40492)
[PASS] test_approve_and_transferFrom_round_trip() (gas: 125958)
[PASS] test_constructor_sets_owner_and_faucet_defaults() (gas: 23713)
[PASS] test_faucetMint_at_exact_cap_succeeds() (gas: 89512)
[PASS] test_faucetMint_blocksDuringCooldown() (gas: 93966)
[PASS] test_faucetMint_cooldown_is_per_address() (gas: 144910)
[PASS] test_faucetMint_credits_caller() (gas: 91957)
[PASS] test_faucetMint_partial_cooldown_still_blocks() (gas: 93847)
[PASS] test_faucetMint_revertsOnZeroAmount() (gas: 16103)
[PASS] test_faucetMint_revertsOverCap() (gas: 18033)
[PASS] test_faucetMint_revertsWhenDisabled() (gas: 18145)
[PASS] test_faucetMint_succeeds_at_exact_cooldown_boundary() (gas: 99636)
[PASS] test_metadata_matches_circle_usdc_abi() (gas: 16794)
[PASS] test_mintMany_credits_all_recipients_in_one_tx() (gas: 124753)
```

```
[PASS] test_mintMany_revertsIfNotOwner() (gas: 16221)
[PASS] test_mintMany_revertsOnEmptyList() (gas: 13345)
[PASS] test_mintMany_revertsOnZeroAddressInList() (gas: 64942)
[PASS] test_mint_credits_recipient_and_increments_totalSupply() (gas: 63566)
[PASS] test_mint_revertsIfNotOwner() (gas: 15831)
[PASS] test_mint_revertsOnZeroAmount() (gas: 15643)
[PASS] test_mint_revertsOnZeroRecipient() (gas: 14043)
[PASS] test_pause_blocks_admin_mint() (gas: 38517)
[PASS] test_pause_blocks_faucetMint() (gas: 38465)
[PASS] test_pause_blocks_mintMany() (gas: 39083)
[PASS] test_pause_does_not_block_transfers() (gas: 116008)
[PASS] test_pause_revertsIfNotOwner() (gas: 13857)
[PASS] test_setFaucetEnabled_revertsIfNotOwner() (gas: 13777)
[PASS] test_setFaucetParams_can_raise_cap_for_stress_test() (gas: 105926)
[PASS] test_setFaucetParams_revertsIfNotOwner() (gas: 14221)
[PASS] test_setFaucetParams_updates_both_knobs() (gas: 21965)
[PASS] test_transferFrom_infinite_allowance_does_not_decrement() (gas: 121128)
[PASS] test_transferFrom_revertsOnInsufficientAllowance() (gas: 95108)
[PASS] test_transferOwnership_fullFlow() (gas: 87398)
[PASS] test_transferOwnership_revertsOnZeroAddress() (gas: 11851)
[PASS] test_transfer_moves_balance() (gas: 93968)
[PASS] test_transfer_revertsOnInsufficientBalance() (gas: 16386)
[PASS] test_transfer_revertsOnZeroDestination() (gas: 66181)
[PASS] test_unpause_restores_mint() (gas: 67330)
Suite result: ok. 38 passed; 0 failed; 0 skipped; finished in 1.88ms (2.39ms CPU time)

Ran 4 test suites in 16.52ms (6.89ms CPU time): 153 tests passed, 0 failed, 0 skipped (153 total tests)
```

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.